

Machine Learning Foundations.

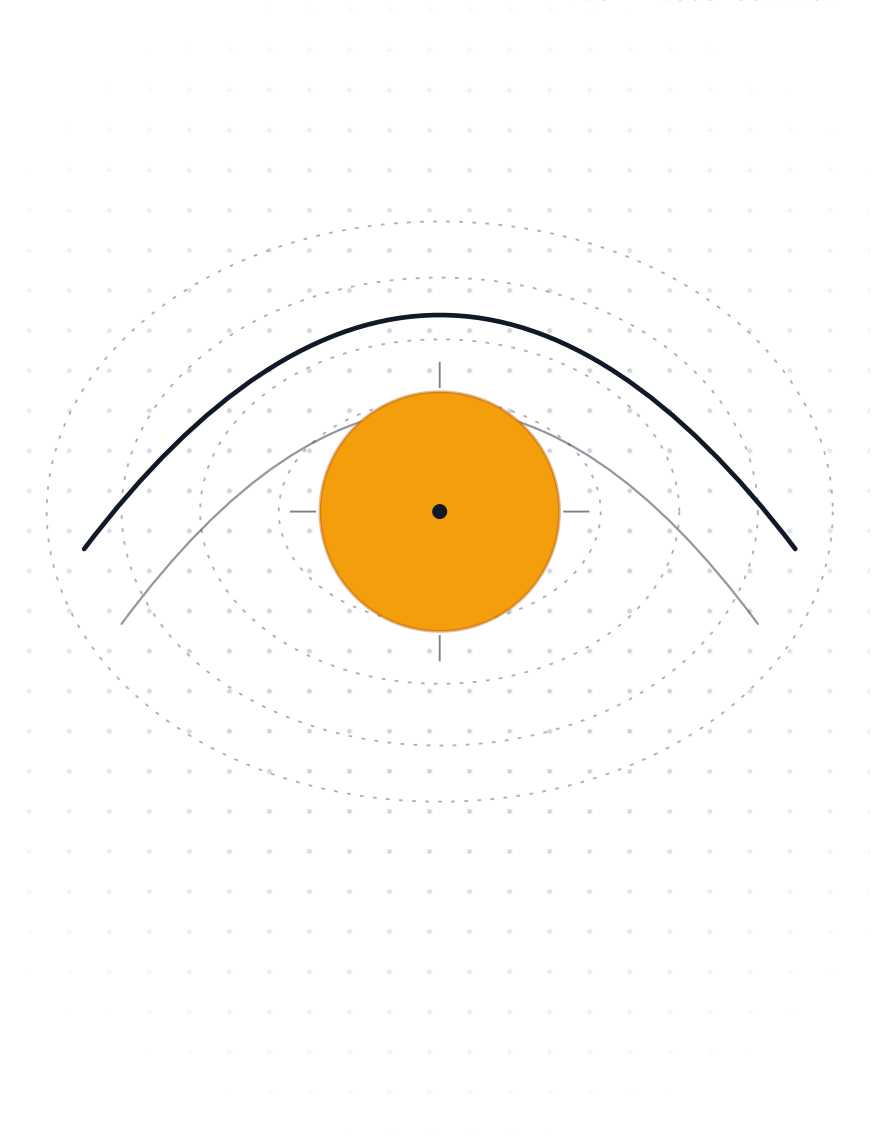
From learning theory to gradient descent, regularization, and logistic regression — seven core ideas, derived and visualized.

Regression

Classification

Optimization

16-SLIDE DECK · THEORY + MATH + CHARTS



Seven ideas that build everything else

Each topic layers on the previous one — start with what learning means, end with logistic regression as a classifier.

3 SECTIONS · 7 TOPICS

Foundations → Optimization → Linear Models

01

Learning from Data

Generalization over memorization; the $X \rightarrow y$ framework.

02

The Core Framework

Loss, cost, empirical risk, train/val/test splits.

03

Bias-Variance Tradeoff

Why capacity is a double-edged sword.

04

Gradient Descent

Walking downhill on the loss surface.

05

Linear Regression

MLE derivation + Normal Equations.

06

Regularization

Ridge shrinks, Lasso zeroes.

07

Logistic Regression

Sigmoid + log-loss = classification.

■ Each topic numbered 01 → 07 · dependency chain: 02 needs 01 · 04 needs 02 · 05-07 need 04

SECTION

01

03 SECTIONS TOTAL • FOUNDATIONS • OPTIMIZATION • LINEAR MODELS

Foundations of Learning

Before models, we need to agree on what learning even means — and how we measure whether a model has actually done it.

Generalization

Loss Functions

Bias-Variance

SLIDES 03 → 07 • 3 TOPICS

Learning is generalization, not memorization

Before ML, programming was deterministic: humans wrote explicit rules. Machine learning flips this — we hand the computer data and answers, and ask it to discover the rules.

A model that merely memorizes training data is useless. True learning means capturing the underlying mathematical structure well enough to predict on data it has never seen.

We assume a true unknown $f(x)$ plus noise ε : $y = f(x) + \varepsilon$. Our hypothesis $\hat{f}(x)$ approximates f as closely as possible.

NOTATION

REFERENCE CARD

X	feature matrix, shape $N \times D$
y	target vector, shape $N \times 1$
$f_{\theta}(x)$	hypothesis parameterized by θ
$\hat{y} = f_{\theta}(x)$	prediction
θ	parameters the model adjusts

Regression

Classification

Unsupervised

How an algorithm knows $\theta=2$ beats $\theta=1$

DEFINITIONS

LOSS · COST · RISK · SPLITS

$L(y, \hat{y})$	Loss — error for a single data point.
$J(\theta)$	Cost / Empirical Risk — average loss over the training set.
ERM	Empirical Risk Minimization — minimize empirical risk because true risk is unknowable.
Train / Val / Test	Splits — Train updates θ ; Val tunes hyperparameters; Test is touched exactly once.

EMPIRICAL RISK

$$J(\theta) = (1/N) \cdot \sum L(y_i, f_{\theta}(x_i))$$

MEAN SQUARED ERROR (MSE)

THE CANONICAL L

Geometrically, $(y_i - \hat{y}_i)$ is the vertical residual between the true point and the prediction. We square it for two reasons:

- 1 Forces all errors positive so +2 and -2 don't cancel.
- 2 Penalizes large errors disproportionately — an error of 4 costs 16× an error of 1.

HAND-CALC

 $y = [2, 4, 6]$
 $\text{Model A } \hat{y} = [1, 2, 3] \rightarrow \text{MSE} = (1+4+9)/3 = 4.67$
 $\text{Model B } \hat{y} = [2, 3, 5] \rightarrow \text{MSE} = (0+1+1)/3 = 0.67$

→ **Model B wins.**

A convex bowl with one global minimum

MSE · EMPIRICAL RISK

Convex MSE Loss Surface — One Global Minimum

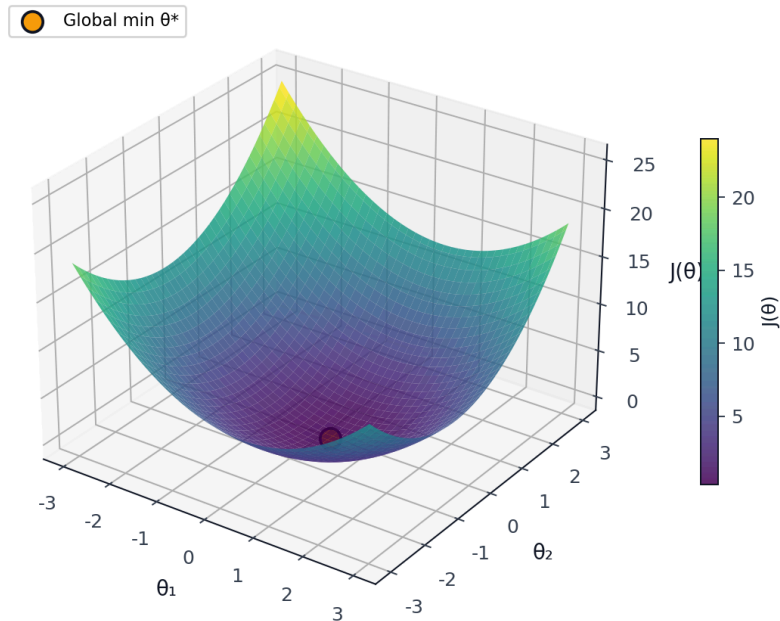


Figure 1. MSE cost $J(\theta)$ over (θ_1, θ_2) . Convexity guarantees gradient descent finds the global minimum.

1 Why MSE is a bowl

For linear regression, $J(\theta)$ is a quadratic form in θ . Its Hessian $X^T X$ is positive semidefinite, so the surface is convex — no local minima to get stuck in.

2 Empirical Risk Minimization

We can't minimize true risk (error over all possible data), so we minimize empirical risk over the training set and pray it generalizes.

3 Three-way split discipline

Train · updates θ
Validation · tunes hyperparameters
Test · locked in a vault until the very end

Why zero training error is a warning, not a victory

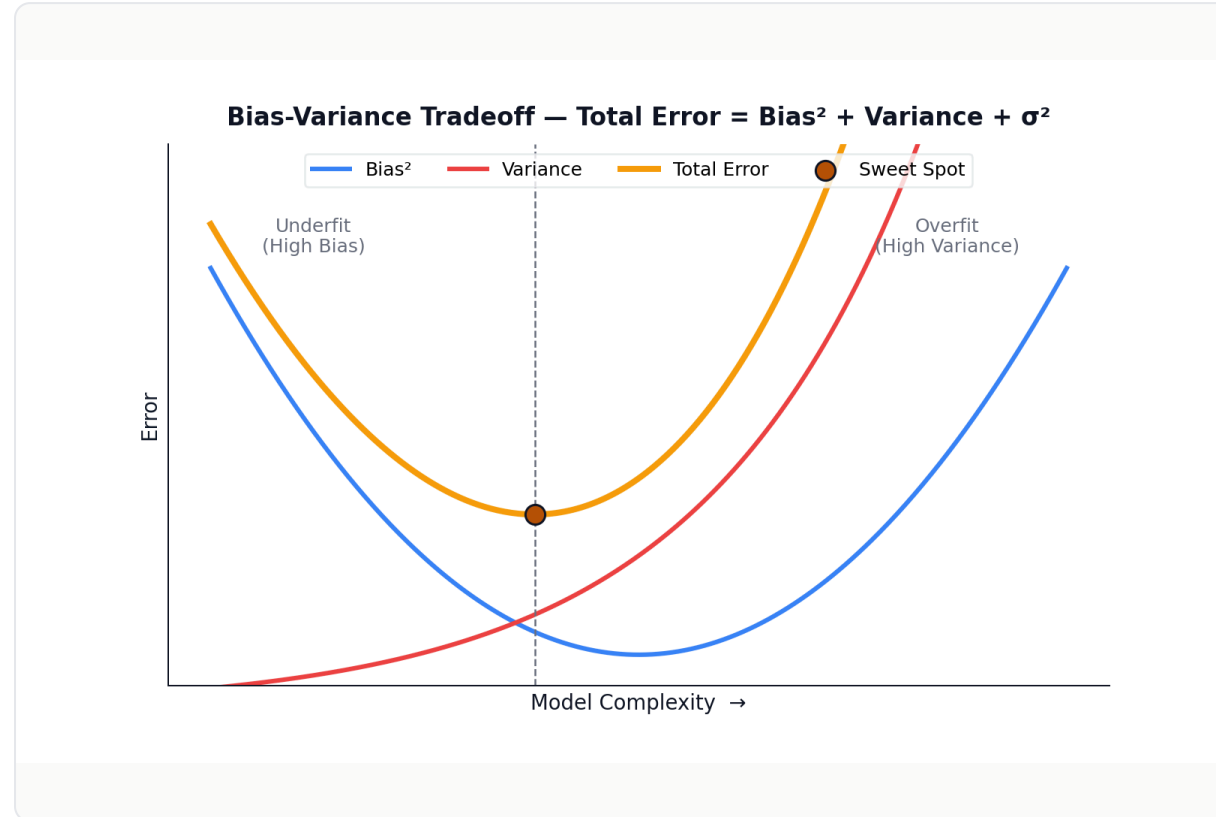


Figure 2. Total error = Bias² + Variance + σ^2 . The sweet spot minimizes the sum, not either term alone.

DECOMPOSITION

$$E[(y - \hat{f}(x))^2] = \text{Bias}^2 + \text{Variance} + \sigma^2$$

Bias² gap between average prediction and true $f(x)$.

Variance how much $\hat{f}$ jumps across different training sets.

σ^2 irreducible noise in the universe.

EMPIRICAL SIGNALS

High train error → High Bias → add features / deeper model.
Low train, high val → High Variance → more data / regularization.

02

03 SECTIONS TOTAL • FOUNDATIONS • OPTIMIZATION • LINEAR MODELS

SECTION 02

Optimization

Bridges the gap between defining a model and actually finding its parameters. We use derivatives to walk downhill on the loss surface.

Gradient

Learning Rate

Mini-batch

Walking downhill, blindfolded

1

▲ Intuition

You are blindfolded in a mountainous terrain, trying to find the lowest valley. You feel the slope under your feet; if it dips right, you step right. You repeat until the ground is flat.

That is gradient descent.

FEEL SLOPE

STEP OPPOSITE

REPEAT

2

The update rule

$$\theta_{t+1} = \theta_t - \alpha \cdot \nabla J(\theta_t)$$

$\nabla J(\theta)$ gradient, direction of steepest ascent.

α learning rate, the size of each step.

We subtract the gradient because we want descent, opposite to ascent.

3

Why it works (Taylor)

$$J(\theta - \alpha \nabla J) \approx J(\theta) - \alpha \cdot \|\nabla J(\theta)\|^2$$

Since $\alpha > 0$ and $\|\nabla J\|^2 \geq 0$, the new cost is strictly less than the old — provided α is small enough.

Convex → global min

Non-convex → local min (often good enough)

Convergence vs divergence — α is everything

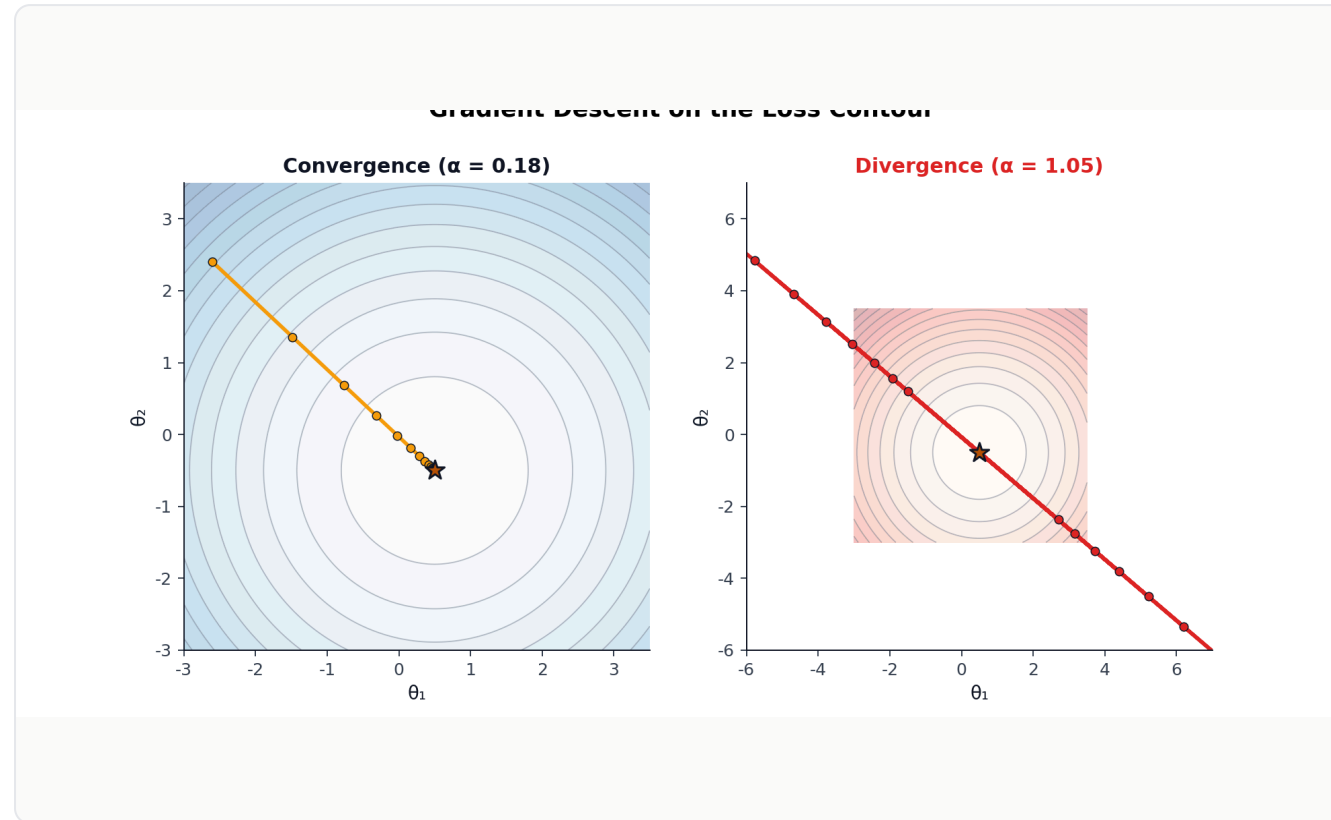


Figure 3. Same loss bowl, two learning rates. Small α converges to the star; large α explodes outward.

WORKED 1D EXAMPLE

$$J(w) = w^2 \cdot \nabla J = 2w \cdot \alpha = 0.1$$

$$w_0=5 \rightarrow w_1=4 \rightarrow w_2=3.2 \rightarrow \dots$$

$$\text{Cost: } 25 \rightarrow 16 \rightarrow 10.24$$

Walking w toward 0, each step shrinks the cost.

THREE FLAVORS OF GD

Batch full dataset per step. Stable, slow.

SGD 1 point per step. Fast, noisy.

Mini-batch 32/128 points. Industry standard.

MODERN REFINEMENTS

Momentum

RMSprop

Adam

Add velocity and adaptive per-parameter step sizes to escape saddle points.

03

03 SECTIONS TOTAL • FOUNDATIONS • OPTIMIZATION • LINEAR MODELS

SECTION 03

Linear Models

Linear regression gives us the cleanest derivation of MSE.
Regularization tames its variance. Logistic regression extends
the same machinery to classification.

Linear

Ridge / Lasso

Logistic

Why MSE? Two derivations, same answer

$$\hat{y} = \theta_0 + \theta_1 x_1 + \dots + \theta_D x_D \quad \cdot \quad \text{vectorized: } \hat{y} = X\theta \quad \cdot \quad J(\theta) = (1/2N) (X\theta - y)^T (X\theta - y)$$

Derivation 1 · Probabilistic (MLE) STATS

Assume y is generated by a true linear function plus Gaussian noise $\epsilon \sim N(0, \sigma^2)$:

$$y_i = \theta^T x_i + \epsilon_i$$

$$P(y_i \mid x_i; \theta) = (1/\sqrt{2\pi\sigma^2}) \cdot \exp(-(y_i - \theta^T x_i)^2 / (2\sigma^2))$$

Maximizing the log-likelihood over the dataset reduces to minimizing $\sum (y_i - \theta^T x_i)^2$.

Profound result: minimizing MSE $\equiv$ MLE under Gaussian noise.

Derivation 2 · Geometric (Normal Equations) LINEAR ALGEBRA

Set the gradient of $J(\theta)$ to zero:

$$\nabla_{\theta} J = X^T X \theta - X^T y = 0$$

$$\Rightarrow \theta = (X^T X)^{-1} X^T y$$

Geometrically, $X\theta$ is the projection of y onto the column space of X ; the residual ($y - X\theta$) is orthogonal to that space.

Matrix inversion is $O(D^3)$ — for huge D , gradient descent wins.

Why L1 produces sparsity but L2 does not

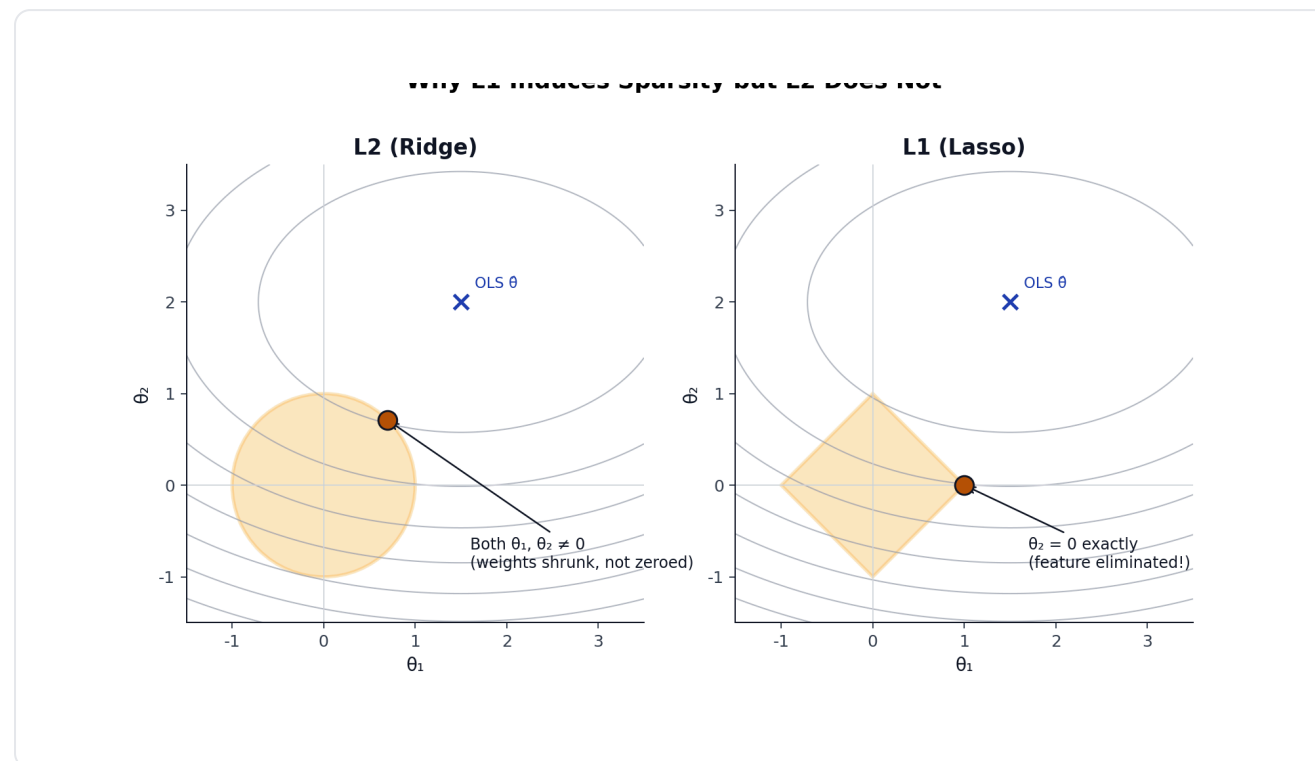


Figure 4. MSE contours (gray) centered at OLS $\hat{\theta}$. The constraint region (amber) touches the contour at the optimum.

The two penalties $J(\theta)$

Ridge (L2): $J(\theta) = \text{MSE} + \lambda \cdot \sum \theta_j^2$

Lasso (L1): $J(\theta) = \text{MSE} + \lambda \cdot \sum |\theta_j|$

λ (alpha in sklearn) is the regularization strength — a gravity well pulling weights toward zero.

The geometric argument **INTUITION**

- L2 constraint is a circle → ellipse usually touches it off-axis → both $\theta_1, \theta_2 \neq 0$ (shrunk).
- L1 constraint is a diamond → corners lie on the axes → ellipse hits a corner → one θ is exactly 0 (eliminated).

Lasso = automatic feature selection.

Ridge = Gaussian prior, Lasso = Laplace prior

MAP estimation BAYESIAN

Maximum Likelihood finds θ that best explains the data. Maximum A Posteriori (MAP) finds θ that is most probable given the data and a prior belief about θ .

Ridge (L2)	← Gaussian prior $N(0, \tau^2)$ on θ →	smooth shrinkage
Lasso (L1)	← Laplace prior (sharp spike at 0) →	exact zeros

Switching from MLE to MAP is exactly what regularization does.

A worked shrinkage example CALC

$$J(\theta) = (1 - 2\theta)^2 + \lambda\theta^2$$
$$dJ/d\theta = 0 \implies \theta = 4 / (8 + 2\lambda)$$

λ	θ
$\lambda = 0$	$\theta = 0.50$
$\lambda = 10$	$\theta = 0.14$
$\lambda = 100$	$\theta = 0.04$

λ acts as a gravity well — bigger λ , smaller θ .

Feature scaling is mandatory.

If feature A is in millions (budget) and feature B is in fractions (rate), their unscaled weights have wildly different magnitudes. Regularization would unfairly punish feature A. Standardize (mean 0, variance 1) before applying Ridge or Lasso.

Squash the linear output, penalize confident wrongness

$$z = \theta^T x \quad \cdot \quad \sigma(z) = 1 / (1 + e^{-z}) \quad \cdot \quad h_{\theta}(x) = P(y=1 \mid x; \theta) = \sigma(\theta^T x)$$

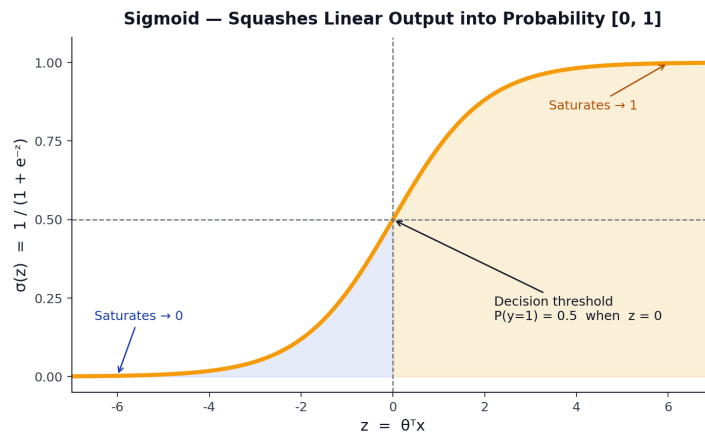


Figure 5. Sigmoid squashes z into $[0, 1]$.

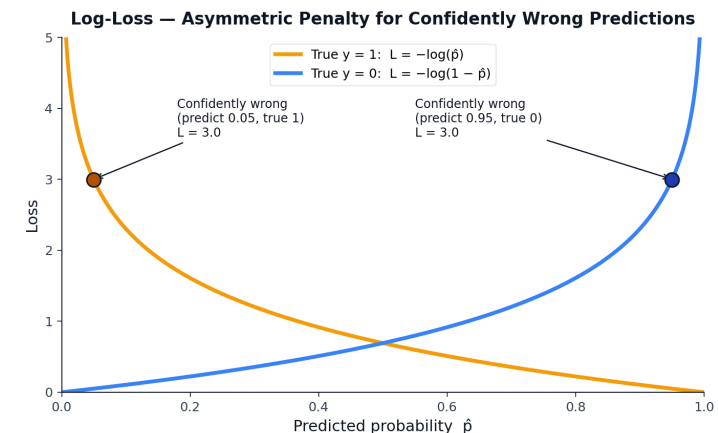


Figure 6. Log-loss explodes as the model is confidently wrong.

WHERE LOG-LOSS COMES FROM

$$J(\theta) = -(1/N) \cdot \sum [y_i \log(h_{\theta}(x_i)) + (1-y_i) \log(1 - h_{\theta}(x_i))]$$

Derived by **MLE on the Bernoulli distribution**. The log makes the product a sum; the negative sign turns maximization into minimization.

Why not MSE on the sigmoid? Because the resulting loss surface is **non-convex** — GD gets stuck. Log-loss on the sigmoid is strictly convex.

Seven ideas, one mental model

FOUNDATIONS · OPTIMIZATION

LINEAR MODELS · WRAP-UP

01

Learning = generalization.

A model that memorizes is useless; aim to capture underlying structure that generalizes to unseen data.

02

Loss is the scorecard.

Cost is average loss over the dataset; ERM minimizes it on the train set and prays it transfers.

03

Bias-Variance is the central tension.

Sweet spot minimizes total error, not either term alone; regularization trades a little bias for a lot of variance.

04

Gradient descent walks downhill.

The update $\theta \leftarrow \theta - \alpha \nabla J$ is provably downhill for small α ; tune α or it diverges.

05

Linear regression has two derivations.

MLE under Gaussian noise justifies MSE; the Normal Equation gives the closed-form $\theta = (X^T X)^{-1} X^T y$.

06

L1 zeroes, L2 shrinks.

The geometry of the constraint — diamond vs circle — explains why Lasso does feature selection and Ridge does not.

07

Logistic regression is classification.

Sigmoid squashes, log-loss punishes confident wrongness, and the math mirrors linear regression's gradient.

**Foundations, complete.**

Every idea here is a stepping stone to neural networks.

NEXT STEPS

implement each from scratch in NumPy · compare with sklearn · revisit with a small real dataset.